

Supplementary Material for: FlexAM: Flexible Appearance-Motion Decomposition for Versatile Video Generation Control

Mingzhi Sheng^{1,*} Zekai Gu^{2,*} Peng Li² Cheng Lin³
Hao-Xiang Guo⁴ Ying-Cong Chen^{1,2,†} Yuan Liu^{2,†}

¹The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

²The Hong Kong University of Science and Technology, Hong Kong SAR, China

³Macau University of Science and Technology, Macau SAR, China

⁴Tsinghua University, Beijing, China

msheng758@connect.hkust-gz.edu.cn, {zekai.gu, plibp}@connect.ust.hk,
chenglin@must.edu.mo, guohaoxiangxiang@gmail.com,
{yingcongchen, yuanly}@ust.hk

1 Additional Details

1.1 Rendering of Motion Video

To visualize the different attribute of motion video, we use a unified rendering procedure as summarized in Alg. 1. The function takes as input a trajectory tensor $\mathbf{P}_{T \times N \times 3}$ and a binary visibility mask $\mathbf{M}_{T \times N}$, together representing a set of 3D point trajectories $\mathbf{p}_i(t) \in \mathbb{R}^3$. Each point $\mathbf{p}_i(t) = (x_i(t), y_i(t), z_i(t))$ corresponds to a specific spatial region. In the case of the identity attribute *fidelity*, we first perform a *color encoding* step on the initial 3D coordinates. For every point i , we normalize its initial position and construct $(x'_i(0), y'_i(0), z'_i(0))$ from $(x_i(0), y_i(0), 1/(z_i(0) + \epsilon))$, where the inverse-depth term and a small constant ϵ avoid numerical instabilities. These normalized components are then directly interpreted as RGB values, yielding a static color code $c_i \in [0, 1]^3$ that uniquely identifies each point and remains fixed over time.

Given these per-point colors, frame generation proceeds identically for all timesteps. At each time t , we first mask out invisible points to obtain $\mathbf{V}_t = \mathbf{P}_t \odot \mathbf{M}_t$, and project the visible 3D coordinates to 2D screen space via $\text{Project2D}(\cdot)$, which converts normalized (x, y) values into integer pixel locations. To correctly handle occlusions, we apply $\text{SortByDepth}(\cdot)$ to order points by their depth (using z) before rendering, and pass the resulting pixel coordinates and colors to the $\text{DrawColorBlocks}(\cdot)$ routine. This subroutine paints fixed-size $k \times k$ color patches (with $k = \text{pointsize}$) at each projected location, yielding a rendered frame I_t . Finally, all frames are encoded into a video through $\text{RenderVideo}(\cdot)$.

*Equal contribution. †Corresponding authors.

Algorithm 1 Rendering of Motion Video ($\mathbf{f}_{\text{identity}}, \mathbf{f}_{\text{freq}}, \mathbf{f}_{\text{depth}}$)

```

1: function RENDERMOTIONVIDEO( $\mathbf{P}_{T \times N \times 3}, \mathbf{M}_{T \times N}, \text{attribute}$ )
2:   Color Encoding (per point):
3:   for  $i = 1, \dots, N$  do
4:      $(x'_i(0), y'_i(0), z'_i(0)) = (x_i(0), y_i(0), \frac{1}{z_i(0) + \epsilon})_{\text{norm} \rightarrow [0,1]^3}$ 
5:     if  $\text{attribute} = \text{IDENTITY}$  then
6:        $c_i \leftarrow (x'_i(0), y'_i(0), z'_i(0))$   $\triangleright \mathbf{f}_{\text{identity}} \in \mathbb{R}^3$ 
7:     else if  $\text{attribute} = \text{FREQ}$  then
8:        $c_i^l \leftarrow (\gamma^l(x'_i(0)), \gamma^l(y'_i(0)), \gamma^l(z'_i(0)))_{\text{norm} \rightarrow [0,1]^3}$   $\triangleright \mathbf{f}_{\text{freq}} \in \mathbb{R}^{12}$ 
9:        $\gamma^l(v) = (\cos(2^l \pi v))_{l \in \{0,1,2,3\}}$ 
10:   Frame-wise Rendering:
11:   for each frame  $t \in [1, T]$  do
12:     if  $\text{attribute} = \text{DEPTH}$  then
13:       for  $i = 1, \dots, N$  do
14:          $c_i(t) \leftarrow \mathcal{S}(z''_i(t))$   $\triangleright \mathbf{f}_{\text{depth}} \in \mathbb{R}^3$ 
15:        $V_t \leftarrow P_t \odot M_t$   $\triangleright$  mask out invisible points
16:        $\text{pixels} \leftarrow \text{Project2D}(V_t^{x,y})$   $\triangleright 3\text{D} \rightarrow 2\text{D}$  pixel coordinates
17:       if  $\text{attribute} \in \{\text{IDENTITY}, \text{FREQ}\}$  then
18:          $\{\text{pix}^\downarrow, c^\downarrow\} \leftarrow \text{SortByDepth}(V_t^z, \{c_i\}_{i=1}^N)$ 
19:       else
20:          $\{\text{pix}^\downarrow, c^\downarrow\} \leftarrow \text{SortByDepth}(V_t^z, \{c_i(t)\}_{i=1}^N)$ 
21:        $\text{DRAWCOLORBLOCKS}(\text{pix}^\downarrow, c^\downarrow)$   $\triangleright$  Render frame
22:   return RenderVideo( $\{I_t\}$ )  $\triangleright$  Assemble frames into attribute-specific videos

```

The rendering of multi-frequency attribute $\mathbf{f}_{\text{freq}}$ reuses this pipeline above and differs only in the color encoding step. Instead of directly mapping the normalized coordinates $(x'_i(0), y'_i(0), z'_i(0))$ to RGB, we first apply a cosine positional encoding function $\gamma^l(\cdot)$ to each coordinate and concatenate the resulting features. For each level of frequency l , The 3-dimensional vector $[\gamma^l(x'_i(0)), \gamma^l(y'_i(0)), \gamma^l(z'_i(0))]$ is then normalized back to $[0, 1]^3$ to obtain a color c_i^l that encodes richer high-frequency spatial information while remaining visually interpretable. As in the identity case, these colors are static over time; only the 3D positions change across frames.

Rather than assigning a fixed color for each tracked point from the initial frame, depth-aware attribute $\mathbf{f}_{\text{depth}}$ compute a per-frame color $c_i(t)$ based solely on the current normalized depth $z''_i(t)$. This scalar depth is mapped through a colormap $\mathcal{S}(\cdot)$ (e.g., Spectral) to produce an RGB value $c_i(t) \in [0, 1]^3$, so that near and far points are visually separated and their depth evolution over time becomes directly observable. The subsequent steps—projection to 2D, depth-based ordering, and drawing colored patches—are identical to the identity and multi-frequency variants.

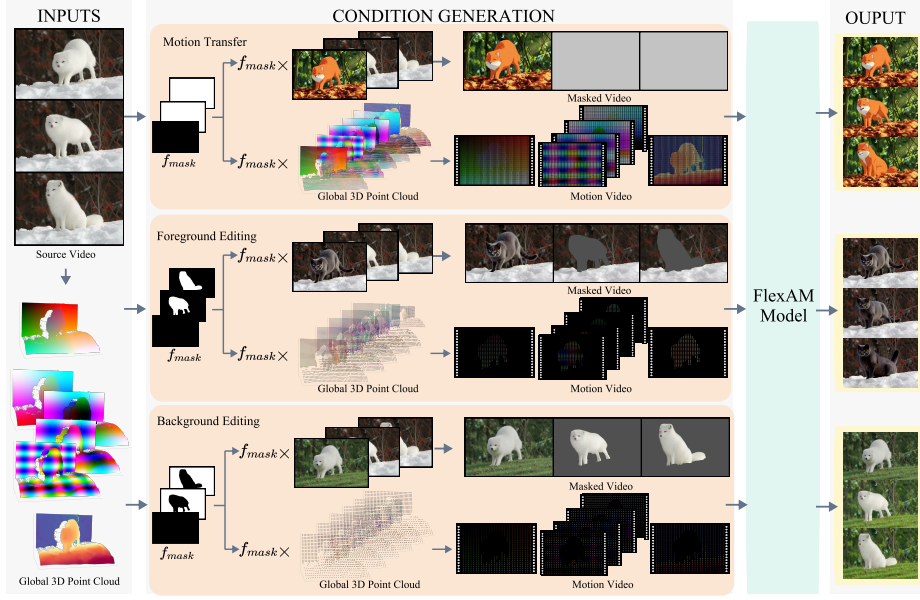


Fig. 1: Overview of condition generation for appearance and motion decomposition. (Top) In **Motion Transfer**, the appearance condition consists of the repainted first frame followed by masked subsequent frames, while the motion video is rendered from the 3D point cloud. (Middle & Bottom) In **Partial Editing** (Foreground/Background), SAM2 is employed to generate a binary mask f_{mask} , which filters both the input video and the 3D point cloud to produce localized appearance and motion conditions. The decoupled signals are then integrated by FlexAM to produce the final result.

Generate Motion Video in Appearance Editing In the appearance editing setting, we take the source video as input and, when performing foreground/background editing, apply the SAM2 [3] model to detect and segment the object to be edited, obtaining a binary mask f_{mask} (for motion transfer, this segmentation step is skipped). We then use DELTA [2] as a 3D tracker to extract the source video’s 3D tracking points $P_{T \times N \times 3}$ and a binary visibility mask $M_{T \times N}$, where the density of tracking points is controlled by a down-sampling factor (with a maximum density of 512×384 points when the factor is set to 1). The schematic construction of the appearance and motion conditions using these elements is illustrated in Fig. 1.

Specifically, the binary mask f_{mask} from SAM2 is used to refine $M_{T \times N}$ by keeping only the tracking points belonging to the target object, while in motion transfer all points are retained. Regarding the appearance condition, we employ Qwen Image Edit [8] to redraw the first frame. In subsequent frames, unedited regions preserve their original pixels, whereas regions to be regenerated are overwritten with a gray value of 127. In foreground editing, this gray region is slightly dilated beyond f_{mask} to accommodate potential shape mismatches between the

redrawn object and the original structure. For objects that are not visible in the first frame but appear later in the video, we use the first visible frame as the appearance reference and start the local editing from that frame. Frames before the object becomes visible are kept unchanged, and the mask and motion video are constructed over the corresponding visible interval.

Generate Motion Video in Camera Control For the image-to-video generation task with a specific camera trajectory, we first use MoGe2 [7] to estimate a depth map from the input image and convert this depth map into a 3D point cloud, yielding initial 3D positions $\mathbf{p}_i(0)$. These 3D points are then reprojected along a prescribed camera trajectory, producing their pixel coordinates $(u_i(t), v_i(t))$ and depth values under each virtual viewpoint. Collecting these per-frame 3D positions results in the trajectory tensor $\mathbf{P}_{T \times N \times 3}$ used as the camera-controlled motion signal.

For the video-to-video camera control task, we first apply DELTA [2] as a 3D tracker to obtain the source video’s 3D tracking points $\mathbf{P}_{T \times N \times 3}$. In parallel, we use VGGT [6] to estimate the camera intrinsics and extrinsics of the input video, thereby recovering its camera parameters and trajectory. Combining the tracked image-space coordinates with the estimated intrinsics and extrinsics yields a 4D dynamic point cloud, where each point has a 3D position in the world coordinate system at every frame. This dynamic point cloud is then reprojected onto a new, user-specified camera trajectory to obtain updated pixel coordinates and depths $(u_i(t), v_i(t), z_i(t))$, forming a camera-controlled version of the original motion.

Generate Motion Video in Spatial Object Editing In the spatial object editing setting, we start from a single input image and use MoGe2 [7] to estimate a depth map, which is then converted into a 3D point cloud $\mathbf{p}_i(0)$. SAM2 [3] is applied to segment the target object, and the resulting object mask is used to select only the 3D points belonging to that object. Given a user-specified object motion trajectory, we manipulate the corresponding object point cloud over time—translating, rotating, or otherwise editing its 3D positions—while keeping the background points fixed. This procedure yields an object-centric dynamic point cloud evolving over time, which we denote as $\mathbf{P}_{T \times N \times 3}$.

1.2 Implementation Details

Training Datasets Our training set comprises 72,617 videos, including 22,931 randomly sampled from OpenVid-1M and the remainder randomly sampled from Koala-36M. The dataset is dominated by three resolutions—1280×720 (58.8%), 576×320 (35.8%), and 768×480 (5.1%)—while all other resolutions (e.g., 1024×1024, 480×384, 720×1280) together constitute only a negligible proportion.

To construct motion video, we render the 3D points estimated by DELTA [2] onto the image plane. These points were rendered as colored rectangles, with their width and height set to 4. To obtain motion controls with varying spatial

sparsity, we evenly partition the training videos into six subsets, each processed with a distinct down-sampling stride (5 to 30) to cover diverse density levels. Combined with density embedding, this multi-level design enables the model to support versatile generative capabilities: it can either achieve fine-grained motion control with high-density signals or perform generalizable motion transfer between objects of different shapes using sparse controls.

Motion-Video Latent Encoding The 18-channel motion video is partitioned into six RGB-like streams and encoded by the pretrained VAE of Wan2.2Fun 5B Control. This design projects the rendered motion representation into the latent space used by the denoising network, allowing the control signal to be integrated without introducing an additional motion-specific encoder. To examine whether the VAE preserves the motion structure, we evaluate its reconstruction quality on 50 RGB videos and 50 motion videos, obtaining PSNRs of 36.8 dB and 34.7 dB, respectively. These comparable reconstruction results indicate that the rendered motion video is effectively preserved after VAE encoding.

Training configurations We jointly optimize the CNN Adapter, density embeddings, and the full DiT backbone of Wan2.2Fun 5B Control [5] for 12,000 steps using the AdamW optimizer on 16 H800 GPUs with a batch size of 16 and a learning rate of 2×10^{-5} .

2 Additional Experimental

We provide additional experimental results and evaluation details for appearance editing, spatial object editing, and human motion editing. These results complement the main paper with further quantitative analyses, evaluation protocols, and robustness analysis.

For fair comparison, all methods are evaluated using the same input videos, reference images or text prompts, editing masks, and preprocessing settings whenever applicable. In particular, the required masks and geometric cues are produced with fixed preprocessing settings and reused across methods whenever they are needed, while each method receives the corresponding control signal required by its own design. This ensures that the comparison is not biased by different preprocessing settings.

2.1 Motion Transfer Stability

We further evaluate the seed stability of FlexAM on motion transfer. In the main paper, all quantitative comparisons are reported using the same random seed for consistency. To further assess the robustness of FlexAM under stochastic sampling, we run FlexAM with five different random seeds and report the mean and variance of the same metrics in Table 1. The small variance across all metrics indicates that the performance of FlexAM remains stable across different random seeds.

Table 1: Seed stability analysis on motion transfer.

Statistic	Tex-Ali $\uparrow$	Tem-Con $\uparrow$	i2v_sub $\uparrow$	i2v_bg $\uparrow$	mot_sm $\uparrow$	aes_q1 $\uparrow$	img_q1 $\uparrow$
Mean	32.44	0.976	0.99	0.99	0.99	0.64	0.68
Var.	1.05×10^{-2}	1.70×10^{-6}	2.90×10^{-7}	1.77×10^{-8}	9.26×10^{-7}	3.25×10^{-5}	3.57×10^{-4}

2.2 Partial Editing

We provide additional quantitative evaluation for partial editing using VBench++ [1] and VE-Bench [4]. Following VBench++, we report subject consistency and background consistency, denoted as i2v_sub and i2v_bg, respectively. We also report the VE-Bench score (VE) to evaluate editing quality. As shown in Table 2, FlexAM consistently outperforms VACE across all metrics, demonstrating stronger preservation and editing performance.

Table 2: Quantitative results on partial editing.

Method	i2v_sub $\uparrow$	i2v_bg $\uparrow$	VE $\uparrow$
VACE	0.957	0.965	1.646
FlexAM (Ours)	0.990	0.993	1.772

2.3 Spatial Object Editing

We provide additional GPT-5-based expert evaluation for spatial object editing, complementing the CLIP-based evaluation in the main paper. The evaluation is conducted on the same translation and rotation cases, with each result scored on a 1–10 scale. We evaluate four dimensions: manipulation accuracy (Manip.), geometric consistency (Geo.), background preservation (Bg.), and overall quality (Overall). As shown in Table 3, FlexAM achieves the best performance across all dimensions, showing stronger spatial manipulation ability and better scene consistency.

Table 3: GPT-5-based expert evaluation on spatial object editing.

Method	Manip. $\uparrow$	Geo. $\uparrow$	Bg. $\uparrow$	Overall $\uparrow$
GeoDiffuser	6.6	4.6	4.6	5.3
DaS	7.6	7.0	5.9	6.8
FlexAM (Ours)	7.8	7.4	6.8	7.4

2.4 Human Motion Editing

We provide an additional qualitative comparison on human motion editing in Fig. 2. Task-specific human-motion priors, such as pose trajectories, can be con-

verted into sparse point trajectories and rendered into the same motion-video format. As shown in Fig. 2, FlexAM better preserves the edited first-frame appearance and source motion under large body movement, while VACE exhibits pose errors and motion artifacts.



Fig. 2: Additional qualitative comparison on human motion editing. FlexAM preserves the edited first-frame appearance and follows the source motion more faithfully under large body movement, while VACE suffers from pose errors and motion artifacts.

2.5 Additional Ablation Study

We provide a feature-level ablation to further analyze the roles of the depth-aware encoding and multi-frequency encoding in the proposed motion video. The depth-aware encoding provides explicit depth information to alleviate ambiguity from similar 2D projections, while the multi-frequency encoding improves the separability of nearby 3D points and reduces motion aliasing.

As shown in Table 4, removing both components leads to the weakest performance. Adding the depth-aware encoding improves reconstruction quality and reduces FVD, while further introducing the multi-frequency encoding achieves the best results across all metrics. These results indicate that the two components provide complementary benefits.

Table 4: Component ablation of the motion video design.

Model Variant	PSNR $\uparrow$	SSIM $\uparrow$	FVD $\downarrow$
FlexAM w/o freq. and depth	18.18	0.6525	811.23
FlexAM w/o freq. encoding	18.56	0.6591	668.98
FlexAM (Full)	19.50	0.6813	485.65

References

1. Huang, Z., Zhang, F., Xu, X., He, Y., Yu, J., Dong, Z., Ma, Q., Chanpaisit, N., Si, C., Jiang, Y., Wang, Y., Chen, X., Chen, Y.C., Wang, L., Lin, D., Qiao, Y., Liu,

- Z.: VBench++: Comprehensive and versatile benchmark suite for video generative models. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025). <https://doi.org/10.1109/TPAMI.2025.3633890>
2. Ngo, T.D., Zhuang, P., Gan, C., Kalogerakis, E., Tulyakov, S., Lee, H.Y., Wang, C.: DELTA: Dense efficient long-range 3d tracking for any video. *arXiv preprint arXiv:2410.24211* (2025)
 3. Ravi, N., Gabeur, V., Hu, Y.T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K.V., Carion, N., Wu, C.Y., Girshick, R., Dollár, P., Feichtenhofer, C.: SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714* (2024)
 4. Sun, S., Liang, X., Fan, S., Gao, W., Gao, W.: VE-Bench: Subjective-aligned benchmark suite for text-driven video editing quality assessment. *arXiv preprint arXiv:2408.11481* (2024)
 5. Wan, T., Wang, A., Ai, B., Wen, B., Mao, C., et al.: Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314* (2025)
 6. Wang, J., Chen, M., Karaev, N., Vedaldi, A., Rupprecht, C., Novotny, D.: Vggt: Visual geometry grounded transformer. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2025)
 7. Wang, R., Xu, S., Dong, Y., Deng, Y., Xiang, J., Lv, Z., Sun, G., Tong, X., Yang, J.: MoGe-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546* (2025)
 8. Wu, C., Li, J., Zhou, J., Lin, J., Gao, K., Yan, K., ming Yin, S., Bai, S., Xu, X., Chen, Y., Chen, Y., Tang, Z., Zhang, Z., Wang, Z., Yang, A., Yu, B., Cheng, C., Liu, D., Li, D., Zhang, H., Meng, H., Wei, H., Ni, J., Chen, K., Cao, K., Peng, L., Qu, L., Wu, M., Wang, P., Yu, S., Wen, T., Feng, W., Xu, X., Wang, Y., Zhang, Y., Zhu, Y., Wu, Y., Cai, Y., Liu, Z.: Qwen-Image technical report. *arXiv preprint arXiv:2508.02324* (2025)